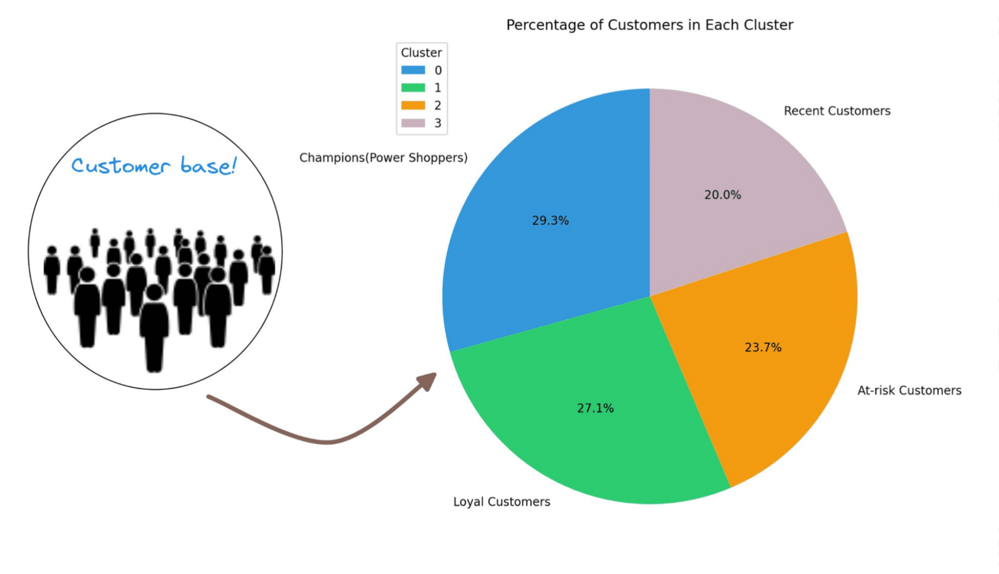
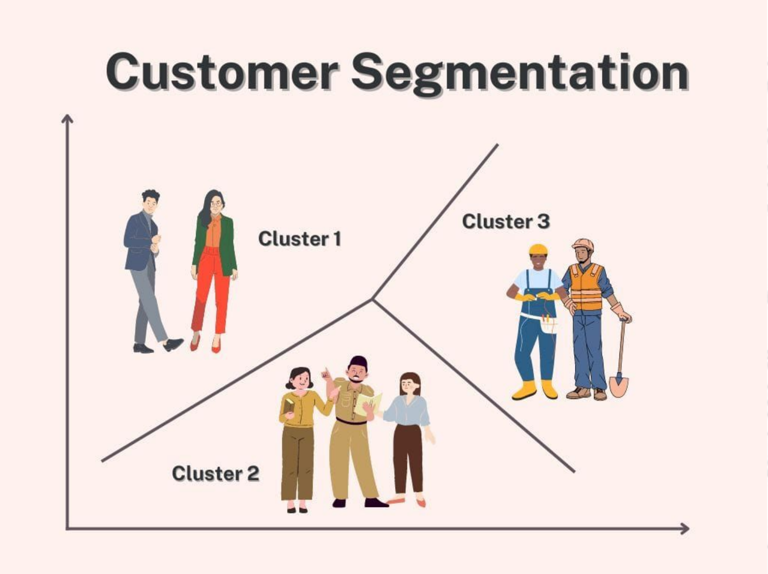
Unsupervised Learning: K Means Clustering and PCA

Abdullah Saihan
Artificial Intelligence Engineer
banglalink

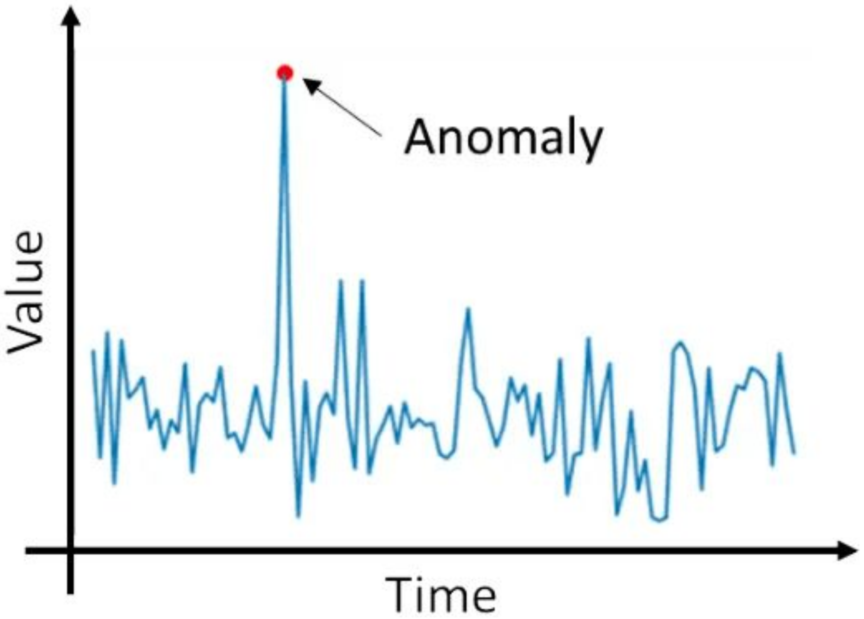
K Means Clustering

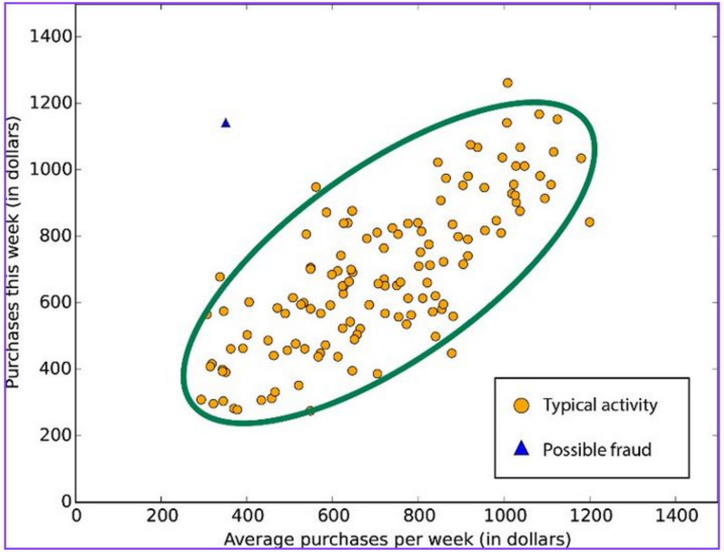
Customer Segmentation



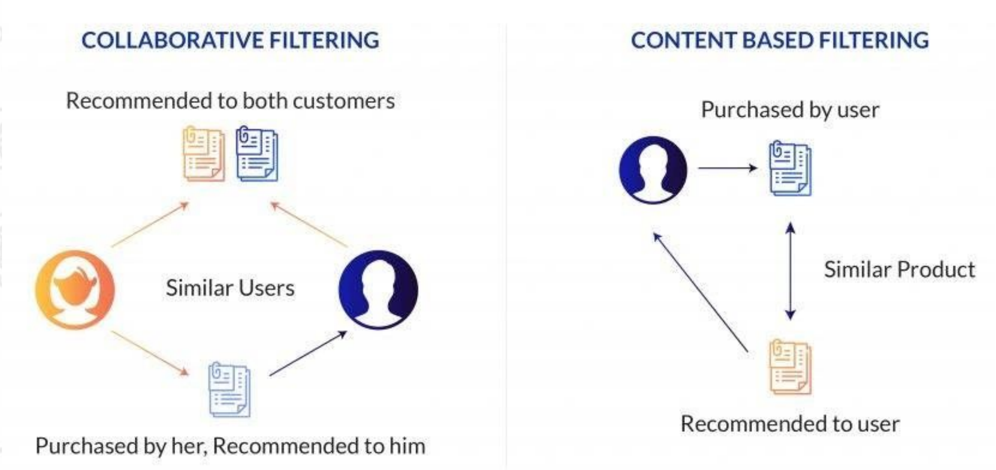


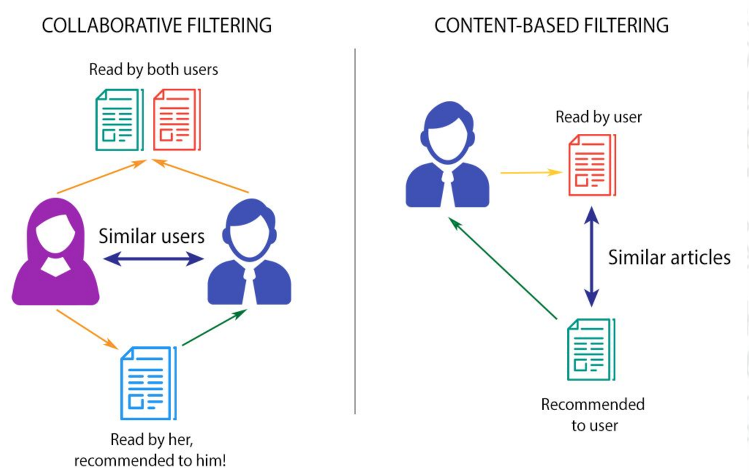
Anomaly Detection





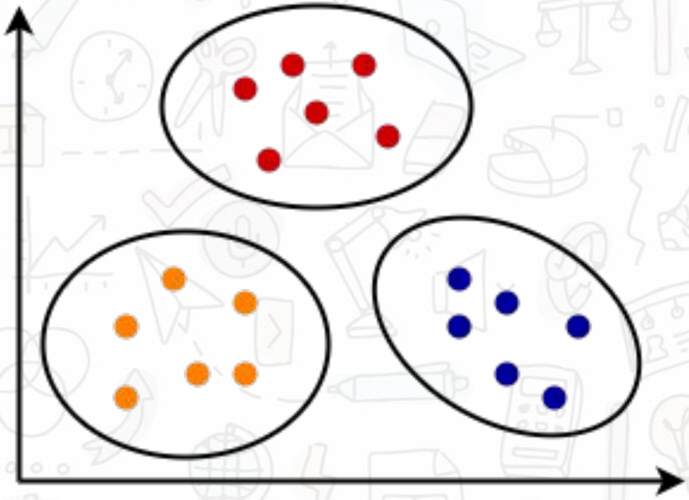
Recommendation Engine





K Means Clustering

Before K-Means



After K-Means

Use Case of K-Means Clustering

01 CUSTOMER SEGMENTATION

Grouping customers based on behavior

02 MARKET RESEARCH

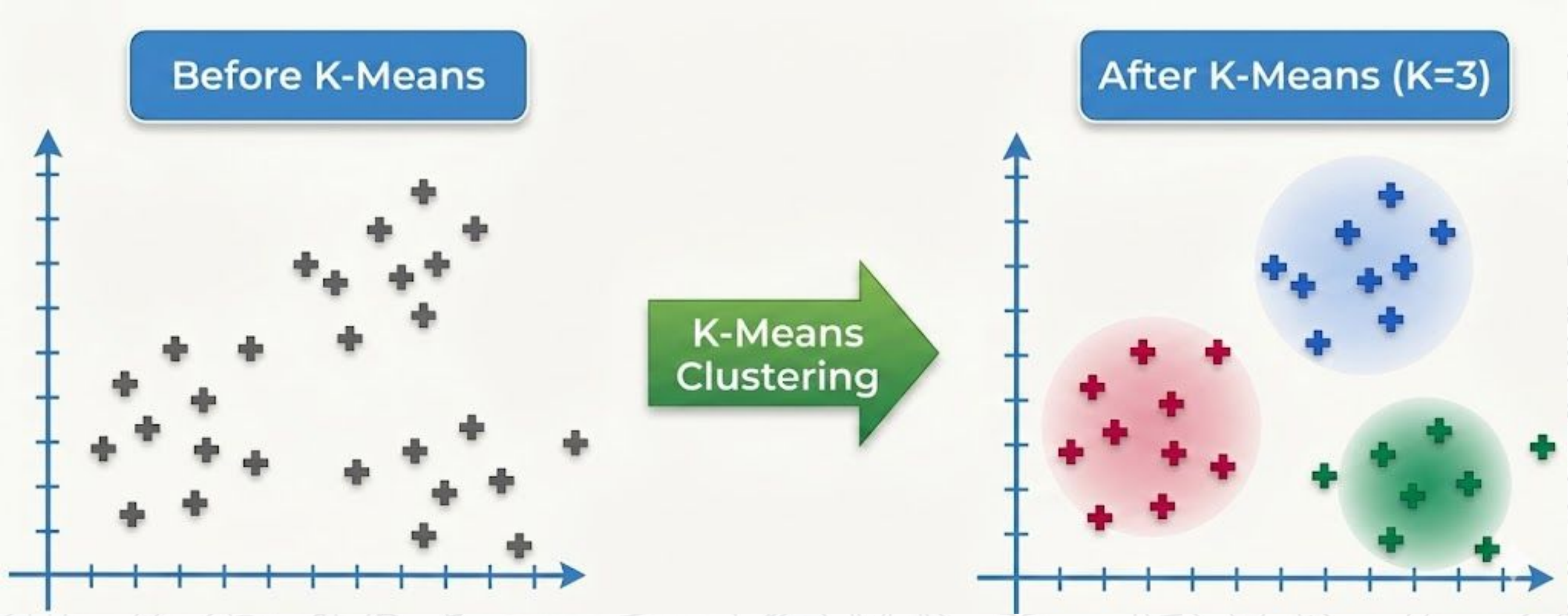
Analyzing market trends and consumer preferences

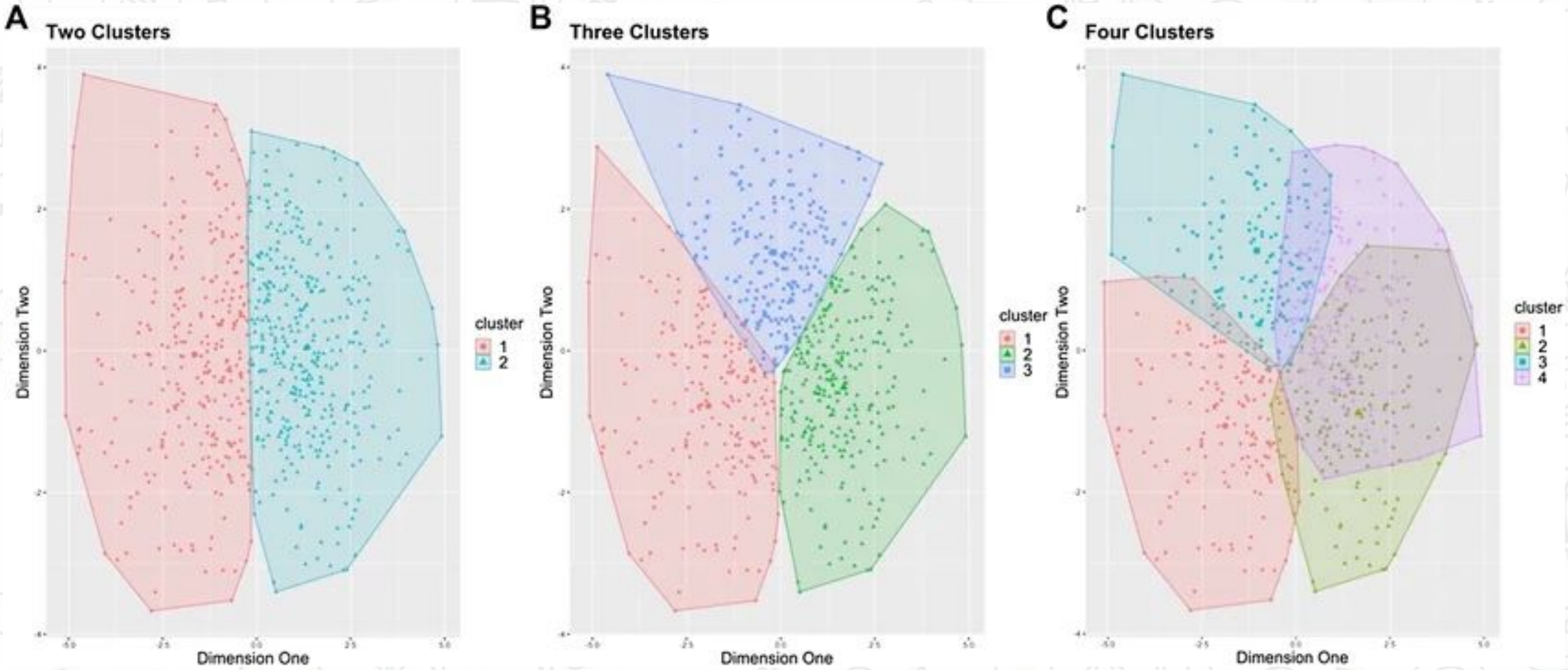
03 IMAGE COMPRESSION

Reducing the size of images

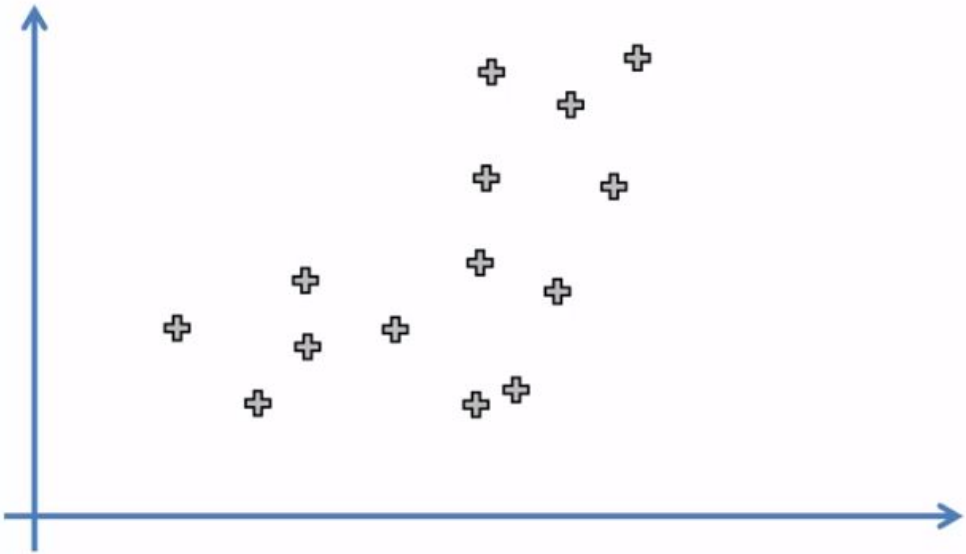
04 ANOMALY DETECTION

Identifying unusual patterns

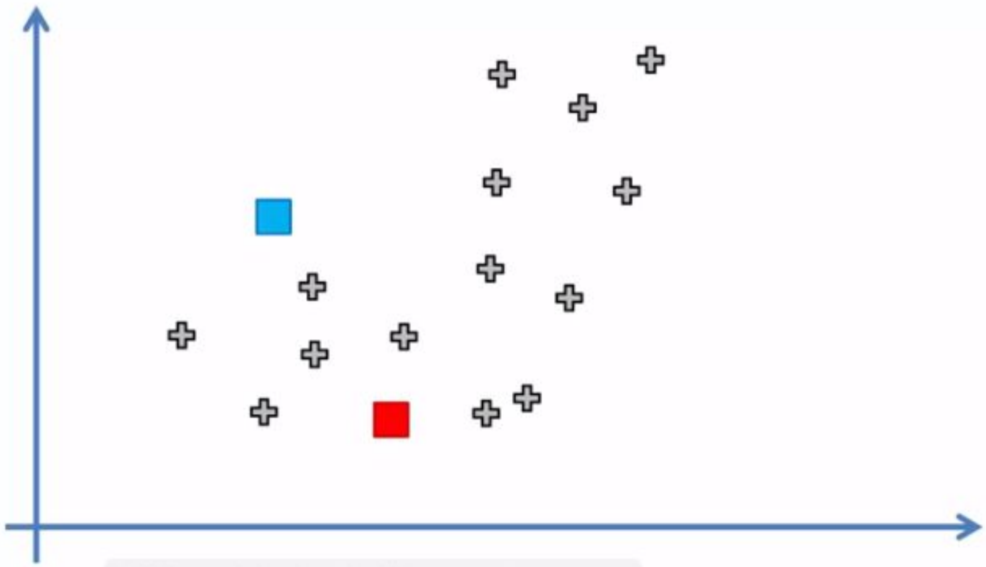

Before K-Means
K-Means Clustering
After K-Means (K=3)

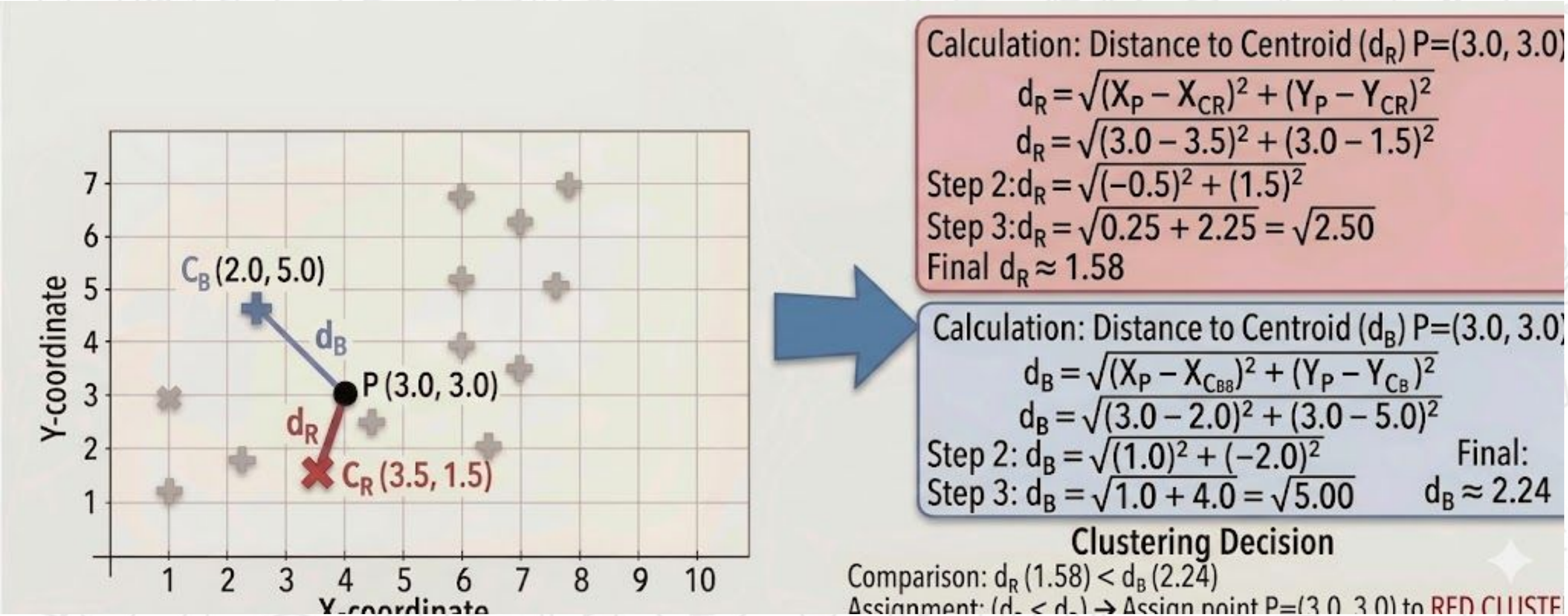

A
Two Clusters
Dimension Two
Dimension One
cluster
1
2
B
Three Clusters
Dimension Two
Dimension One
cluster
1
2
3
C
Four Clusters
Dimension Two
Dimension One
cluster
1
2
3
4

STEP 1: Choose the number K of clusters: K = 2



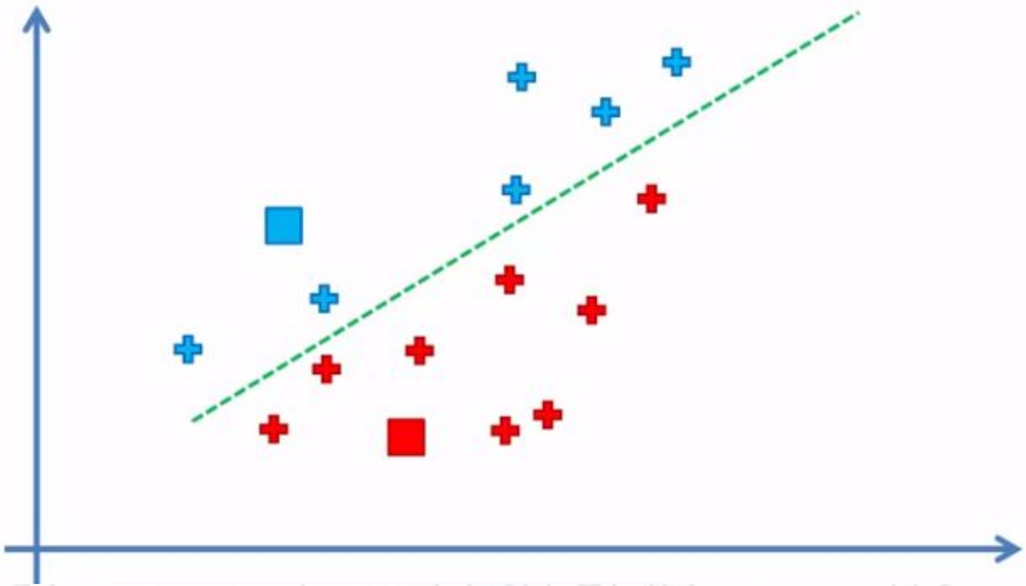
STEP 2: Select at random K points, the centroids (not necessarily from your dataset)



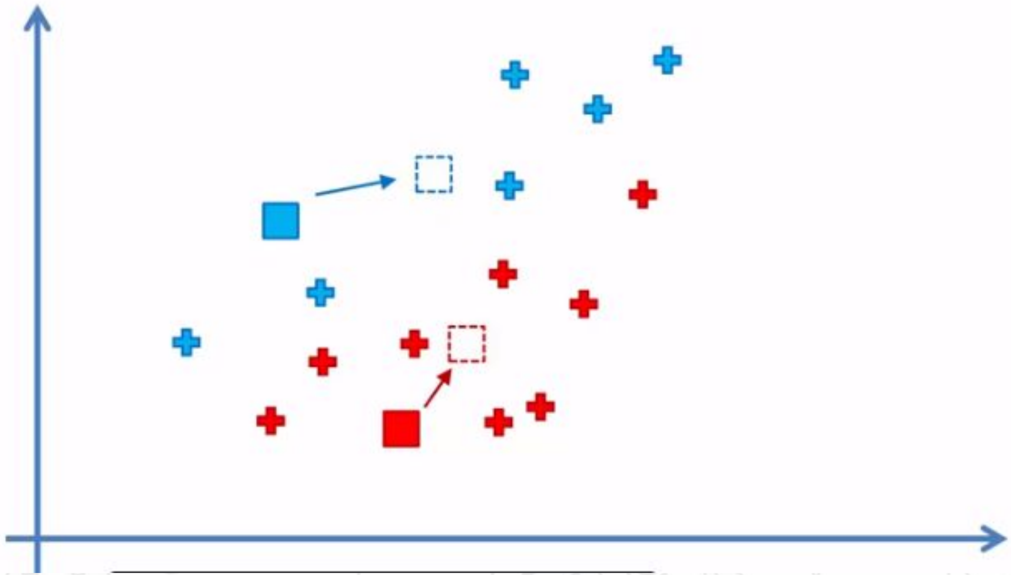


Y-coordinate
X-coordinate
C_B (2.0, 5.0)
d_B
P (3.0, 3.0)
d_R
C_R (3.5, 1.5)
Calculation: Distance to Centroid (d_R) P=(3.0, 3.0)
$d_R = \sqrt{(X_P - X_{CR})^2 + (Y_P - Y_{CR})^2}$
$d_R = \sqrt{(3.0 - 3.5)^2 + (3.0 - 1.5)^2}$
Step 2: $d_R = \sqrt{(-0.5)^2 + (1.5)^2}$
Step 3: $d_R = \sqrt{0.25 + 2.25} = \sqrt{2.50}$
Final $d_R \approx 1.58$
Calculation: Distance to Centroid (d_B) P=(3.0, 3.0)
$d_B = \sqrt{(X_P - X_{CB})^2 + (Y_P - Y_{CB})^2}$
$d_B = \sqrt{(3.0 - 2.0)^2 + (3.0 - 5.0)^2}$
Step 2: $d_B = \sqrt{(1.0)^2 + (-2.0)^2}$
Step 3: $d_B = \sqrt{1.0 + 4.0} = \sqrt{5.00}$
Final: $d_B \approx 2.24$
Clustering Decision
Comparison: d_R (1.58) < d_B (2.24)

STEP 3: Assign each data point to the closest centroid ➡ That forms K clusters



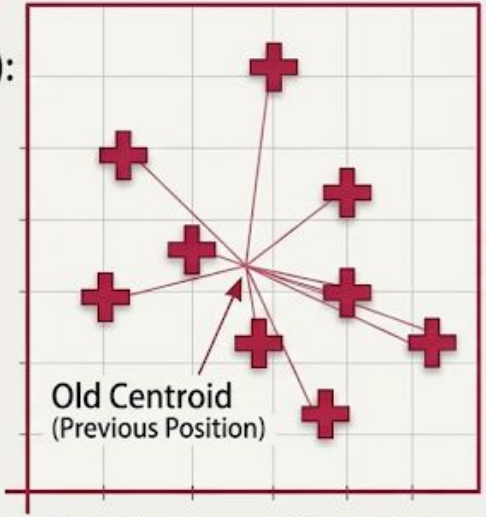
STEP 4: Compute and place the new centroid of each cluster



Ostad

Data Points in Red Cluster (x, y):

- P1: (2, 3)
- P2: (3, 2)
- P3: (4, 4)
- P4: (3, 6)
- P5: (5, 3)
- P6: (1, 5)
- P7: (6, 2)
- P8: (4, 1)



Find Average of Coordinates

Average X-coordinate:

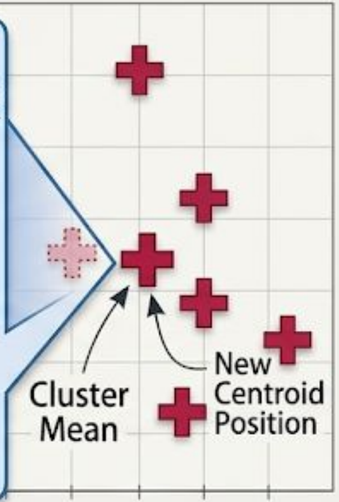
$$X_{avg} = \frac{2+3+4+3+5+1+6+4}{8} = \frac{28}{8} = 3.5$$

Average Y-coordinate:

$$Y_{avg} = \frac{3+2+4+6+3\ 5+2+1}{8} = \frac{26}{8} = 3.25$$

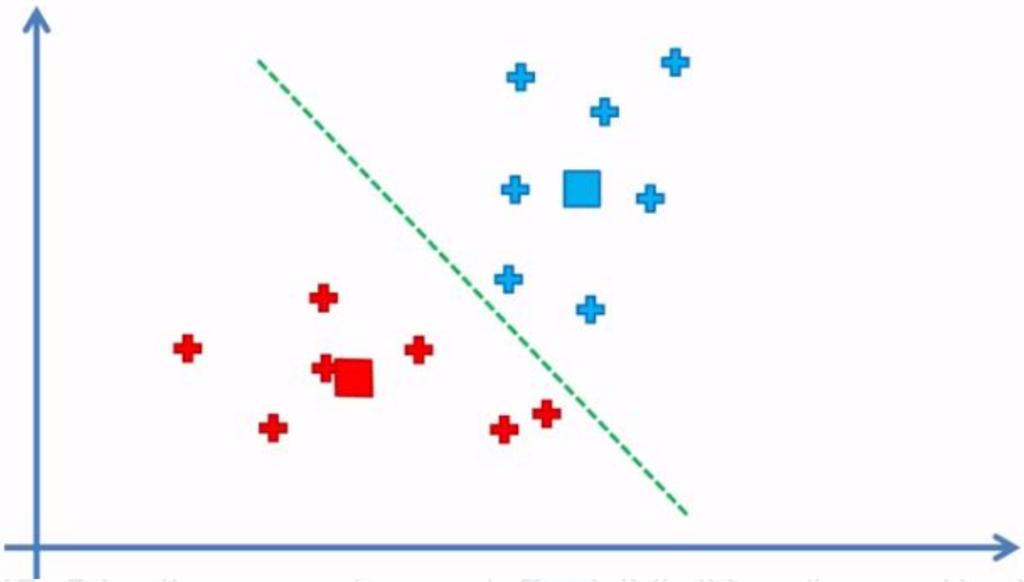
New Centroid Position:

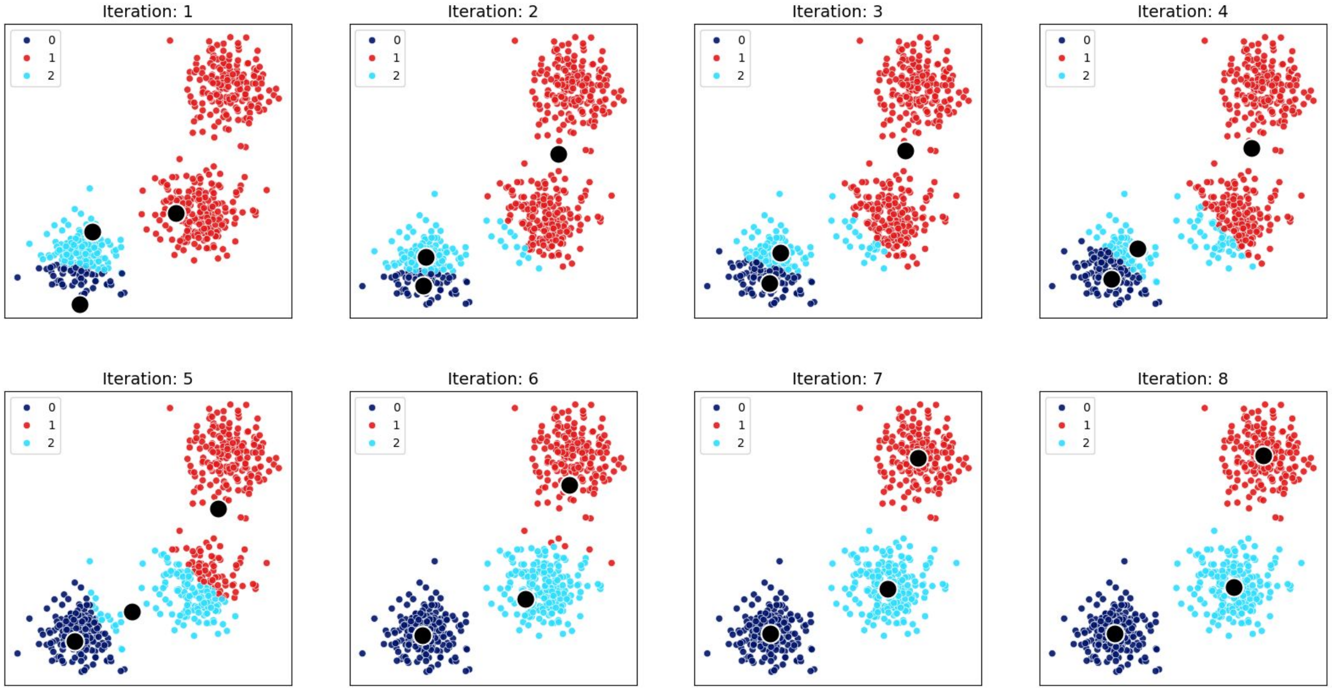
(3.5, 3.25)

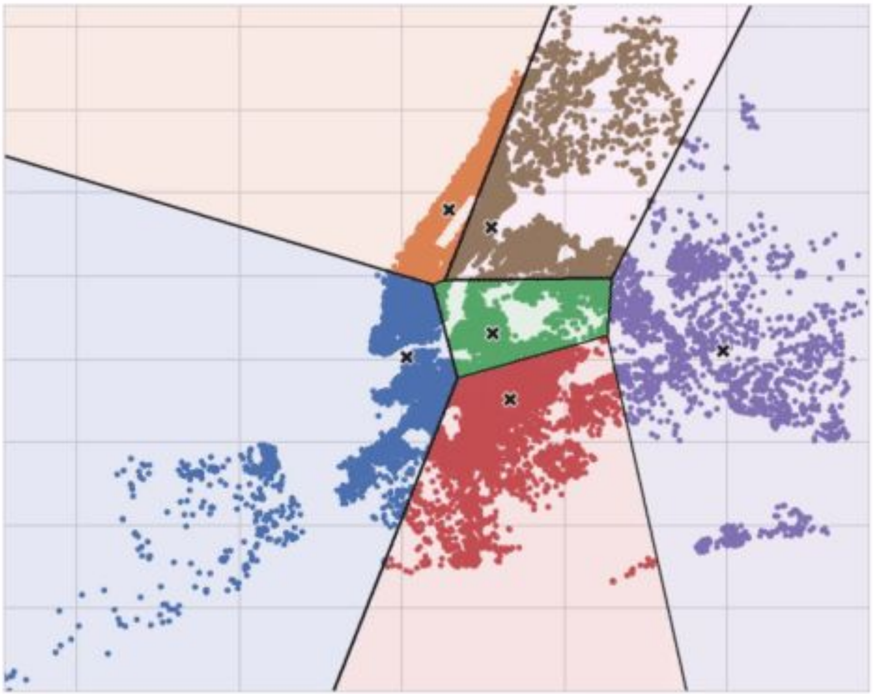


$$\text{New Centroid} = \frac{(\sum(x_i), \sum(y_i))}{N}$$

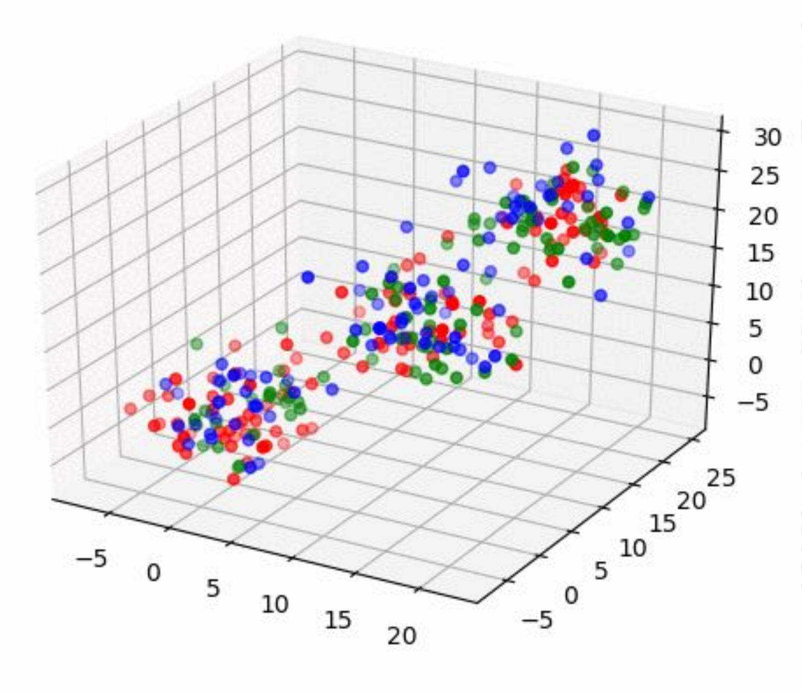
STEP 5: Reassign each data point to the new closest centroid.
If any reassignment took place, go to STEP 4, otherwise go to FIN.




Iteration: 1
Iteration: 2
Iteration: 3
Iteration: 4
Iteration: 5
Iteration: 6
Iteration: 7
Iteration: 8
0
1
2



Kmeans in 3D



The **convergence criteria of K-Means clustering** determine when the algorithm should stop iterating. K-Means usually converges when cluster assignments stop changing or the centroids no longer move significantly.

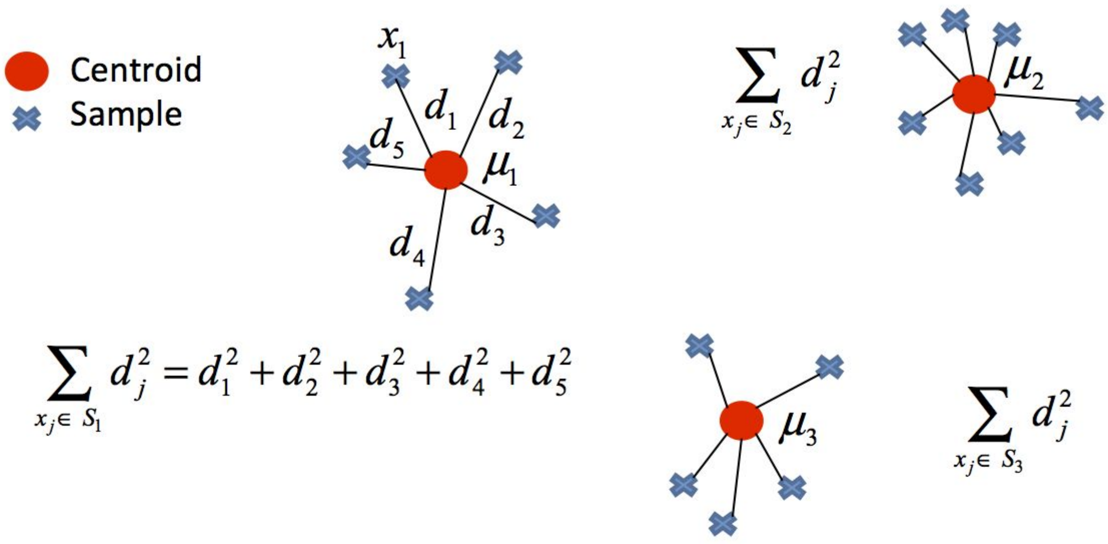
Common convergence criteria:

1. **No change in cluster assignments**
   - Each data point remains in the same cluster as the previous iteration.
   - This is the most common stopping condition.
2. **Centroids stop moving**
   - If the movement of centroids becomes very small (below a threshold ϵ).
   - Example:

   $$\|\mu_t - \mu_{t-1}\| < \epsilon$$
3. **Maximum number of iterations reached**
   - Used to avoid infinite loops.
   - Example: stop after 100 or 300 iterations.
4. **Minimal decrease in objective function**
   - K-Means minimizes the within-cluster sum of squares (WCSS / inertia).

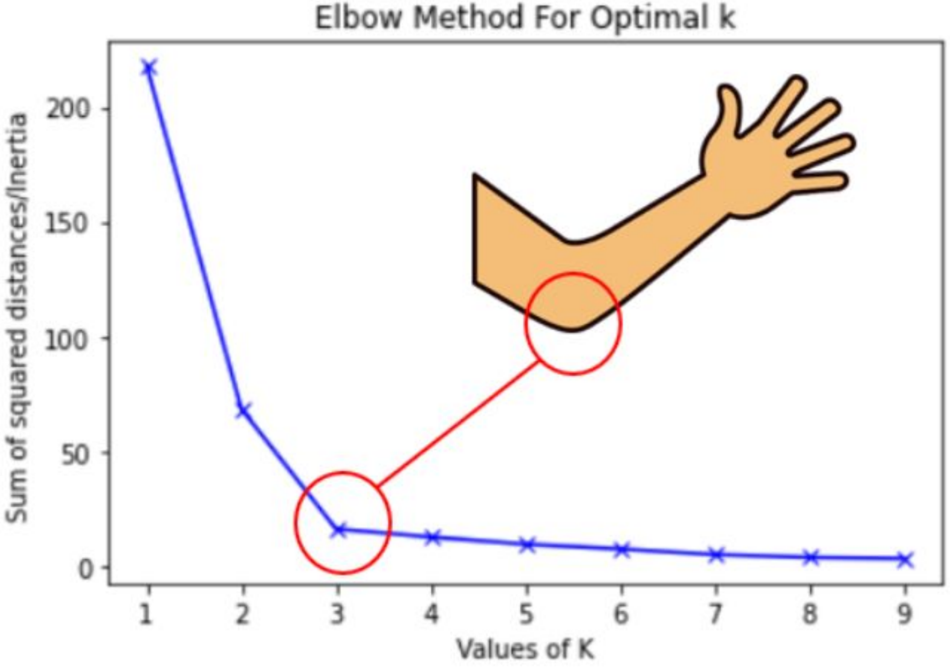
   $$J = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$
   - If improvement becomes negligible, the algorithm stops.

Objective: Minimize WCSS

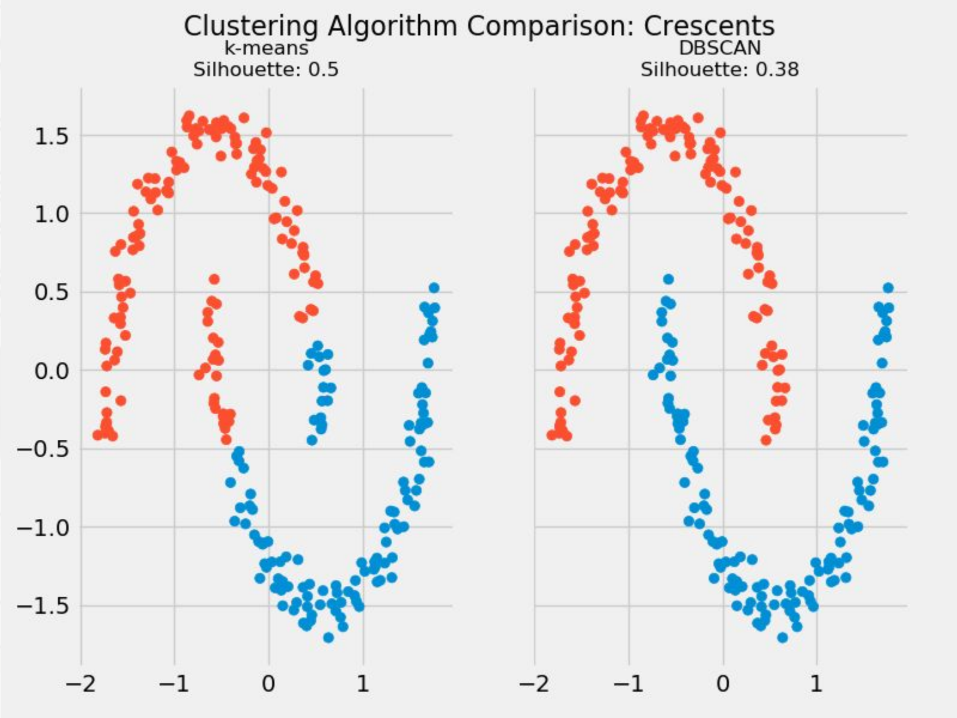


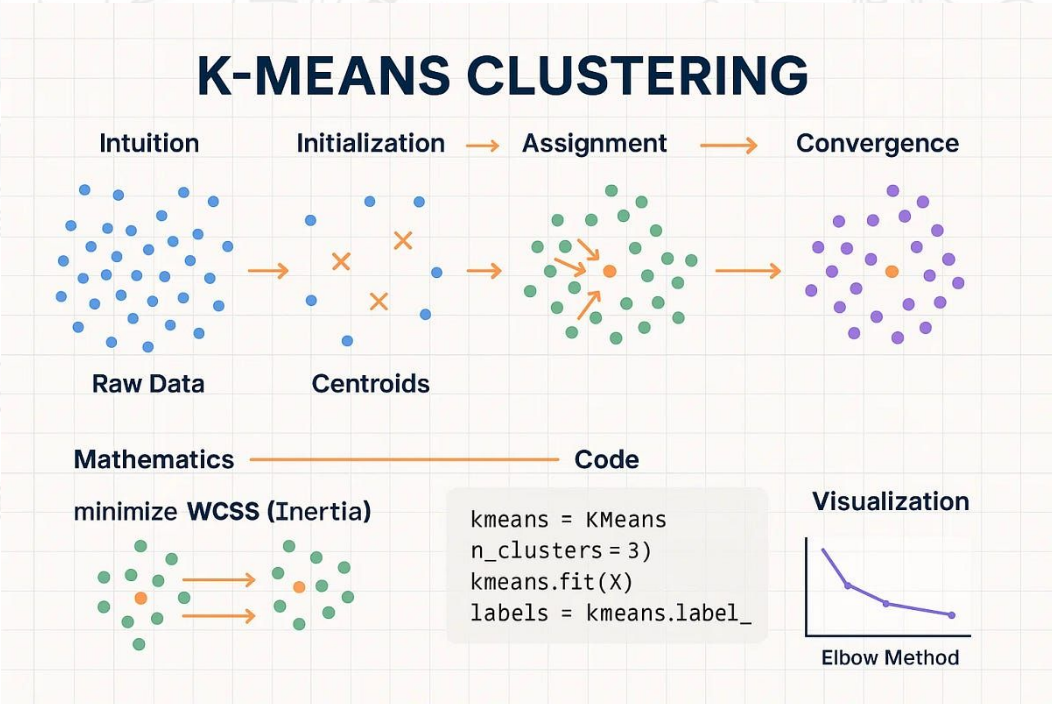
$$\min_S E(\mu_i) = \sum_{x_j \in S_1} d_j^2 + \sum_{x_j \in S_2} d_j^2 + \sum_{x_j \in S_3} d_j^2$$

Finding Optimal K

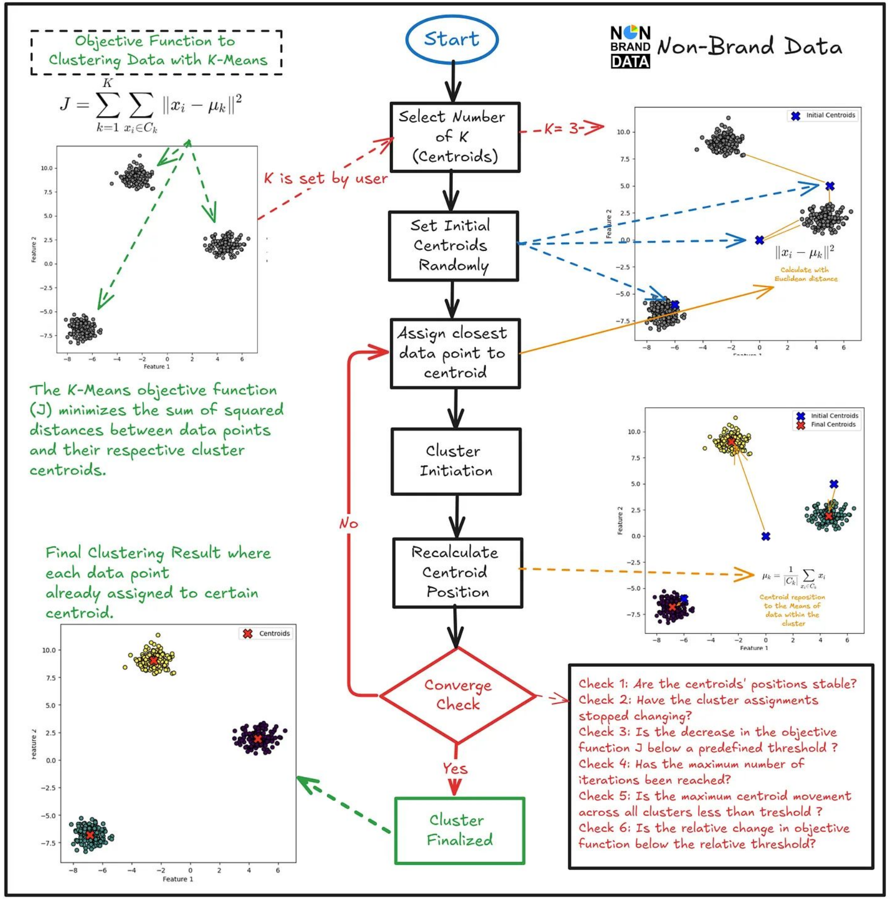


Line plot between K and inertia


Clustering Algorithm Comparison: Crescents
k-means
Silhouette: 0.5
DBSCAN
Silhouette: 0.38
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2
−1
0
1

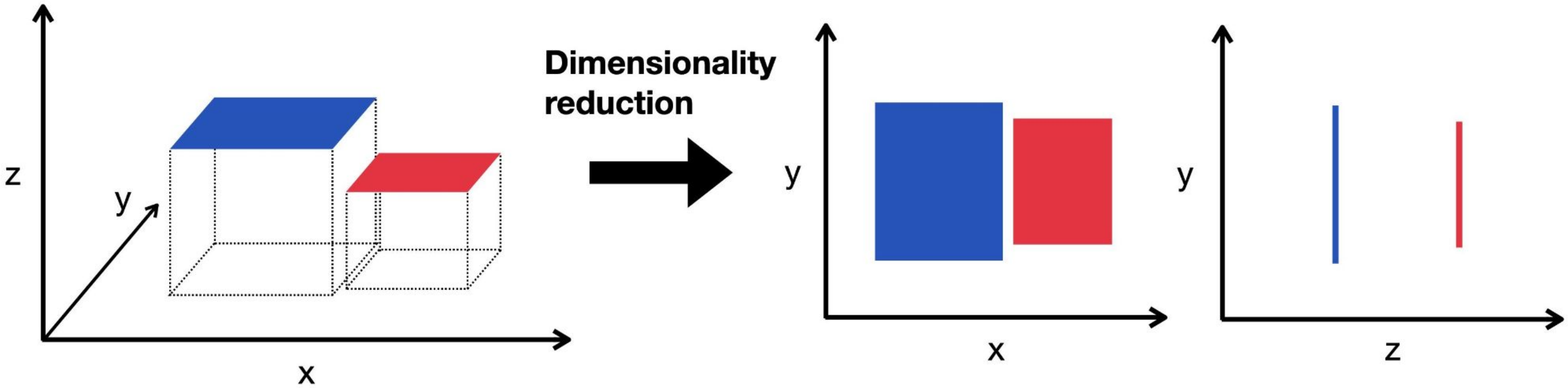

K-MEANS CLUSTERING
Intuition
Initialization
Assignment
Convergence
Raw Data
Centroids
Mathematics
Code
minimize WCSS (Inertia)
kmeans = KMeans
n_clusters = 3)
kmeans.fit(X)
labels = kmeans.label_
Visualization
Elbow Method

How K-Means Works: A Summary

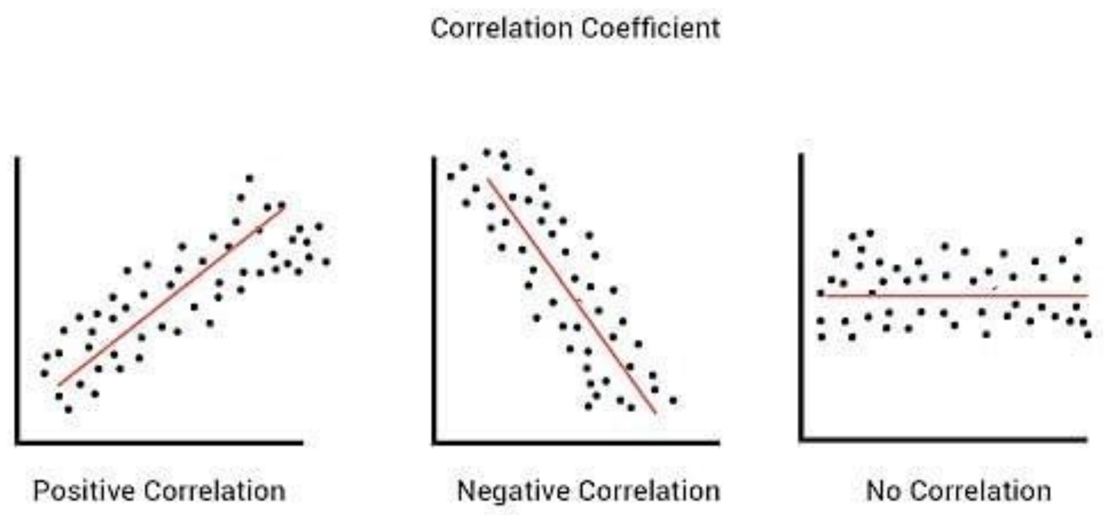


Principal Component Analysis

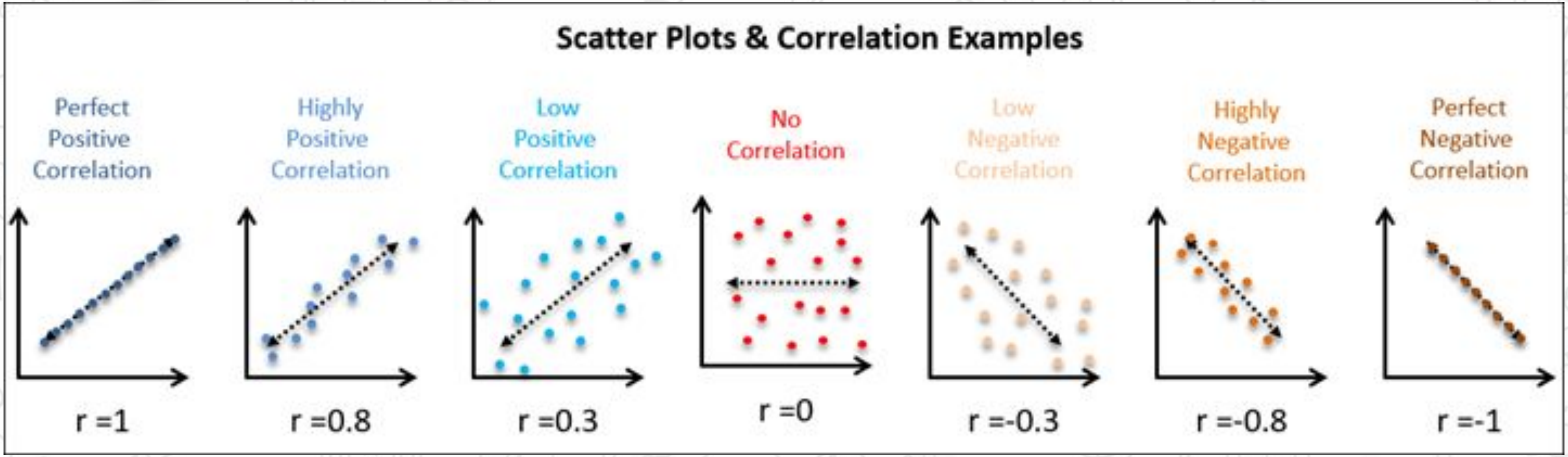
Dimensionality Reduction



Correlation



Correlation



High Variance vs Low Variance

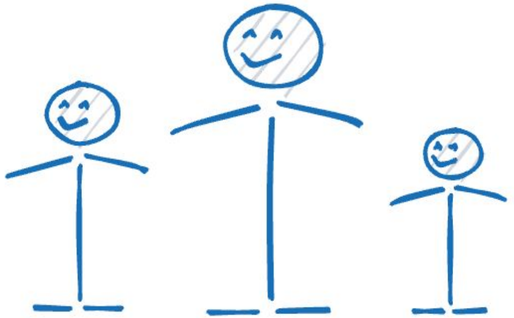
Person	Height
A	145
B	160
C	185

Person	Height
D	172
E	173
F	171

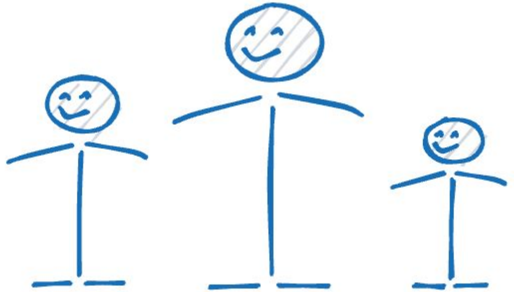
High vs Low Variance Feature

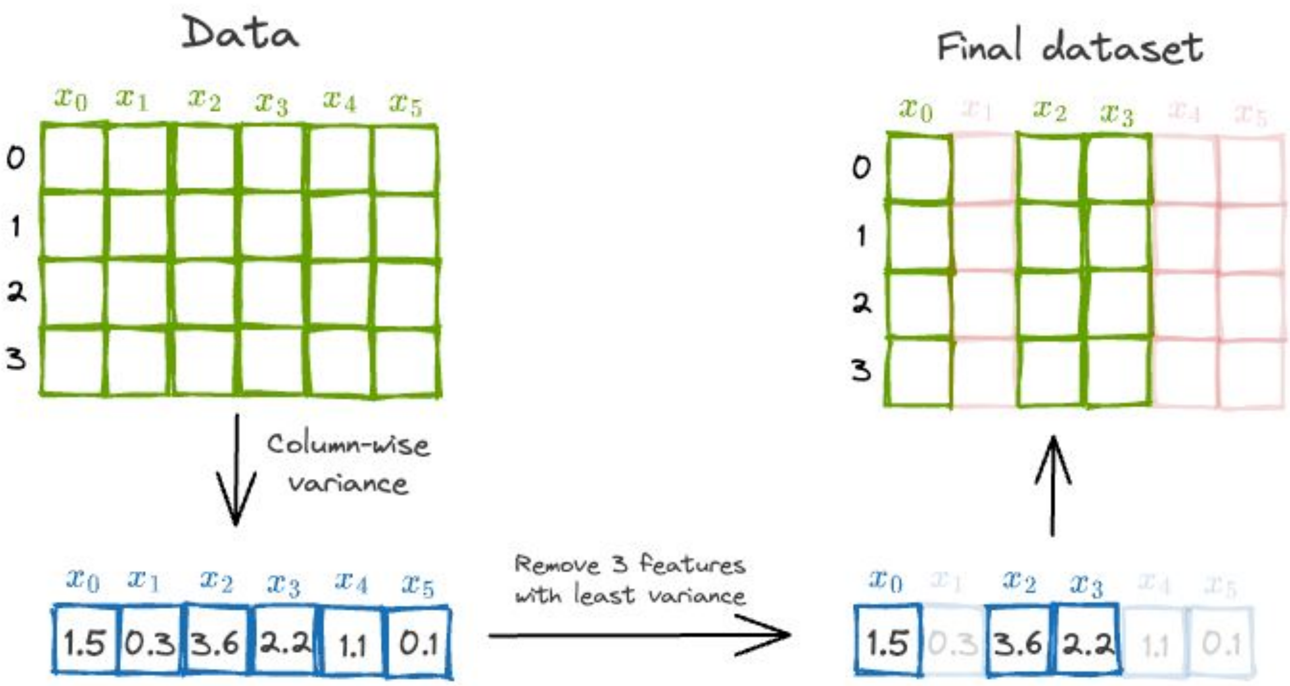
	Height (cm)	Weight (kg)
Andrew	150	71
Nick	170	73
Jonas	140	70

	Height (cm)
Andrew	150
Nick	170
Jonas	140

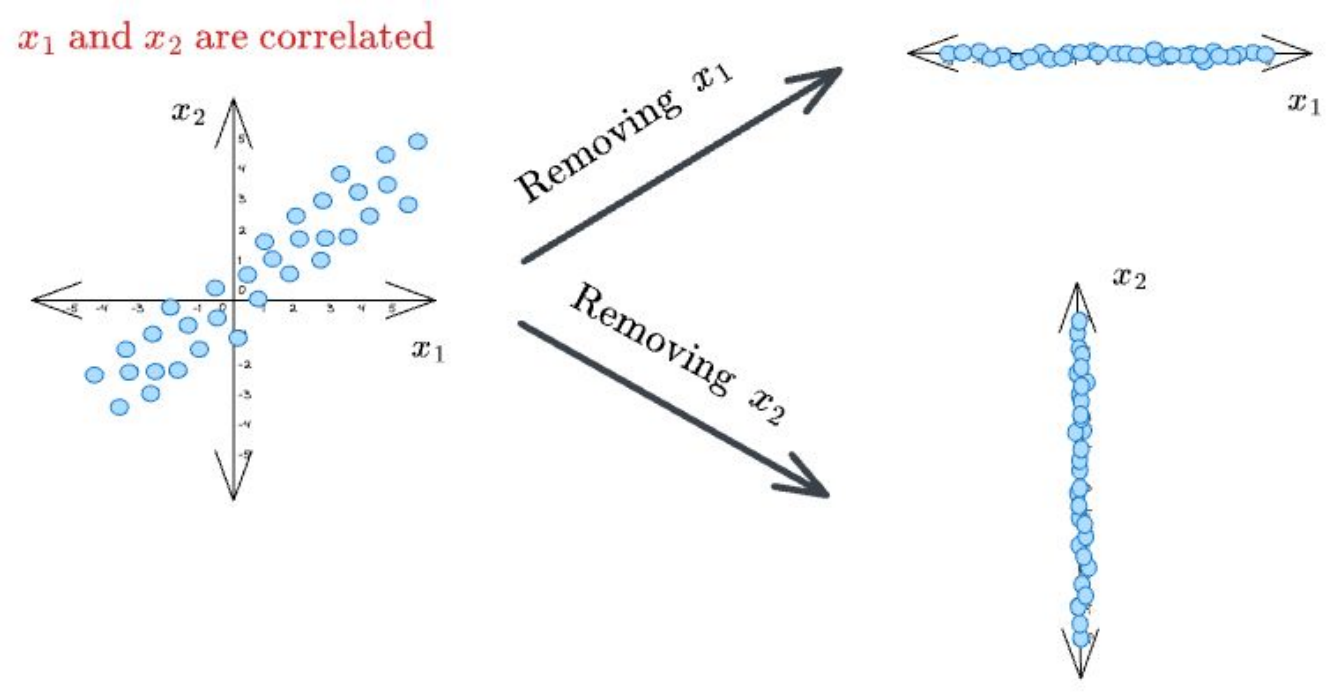


	Weight (kg)
Andrew	71
Nick	73
Jonas	70

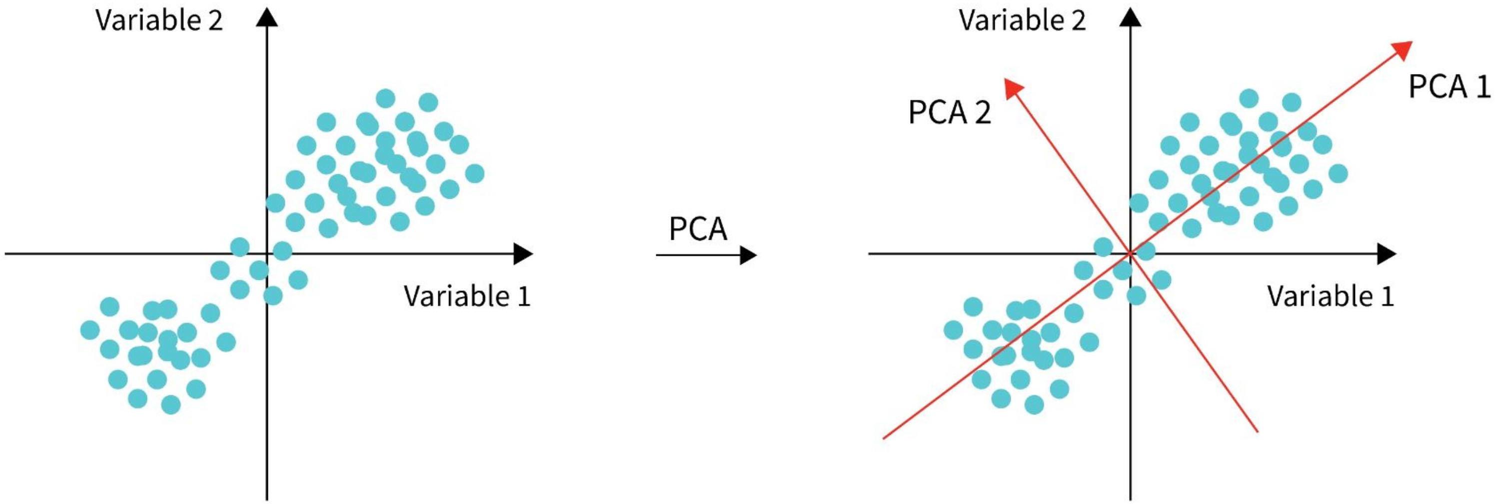


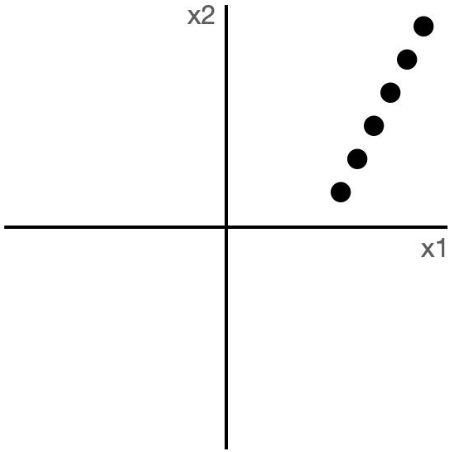


Data
x_0 x_1 x_2 x_3 x_4 x_5
0
1
2
3
Column-wise variance
x_0 x_1 x_2 x_3 x_4 x_5
1.5 0.3 3.6 2.2 1.1 0.1
Remove 3 features with least variance
x_0 x_1 x_2 x_3 x_4 x_5
1.5 0.3 3.6 2.2 1.1 0.1
Final dataset
x_0 x_1 x_2 x_3 x_4 x_5
0
1
2
3

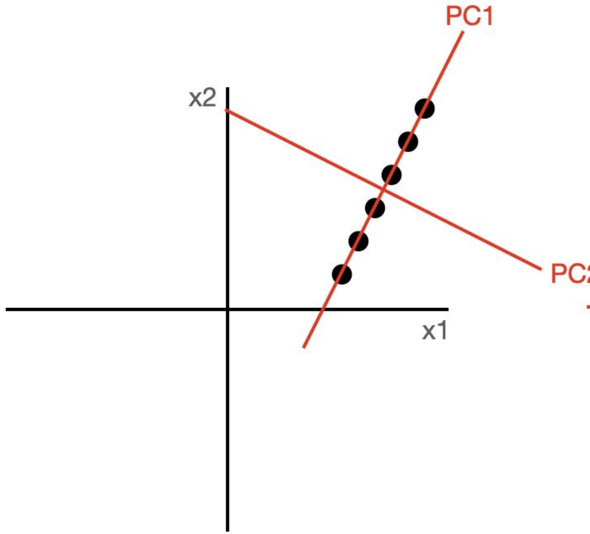


x_1 and x_2 are correlated
x_2
x_1
Removing x_1
Removing x_2
x_1
x_2

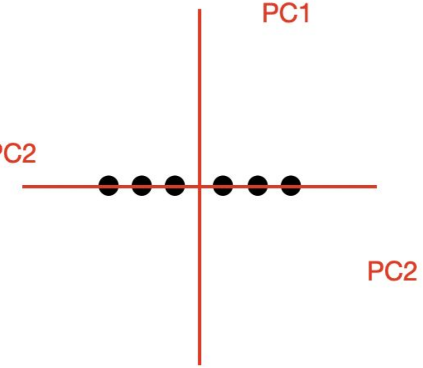

Variable 2
Variable 1
PCA
Variable 2
PCA 1
PCA 2
Variable 1



correlated data in native 2D space

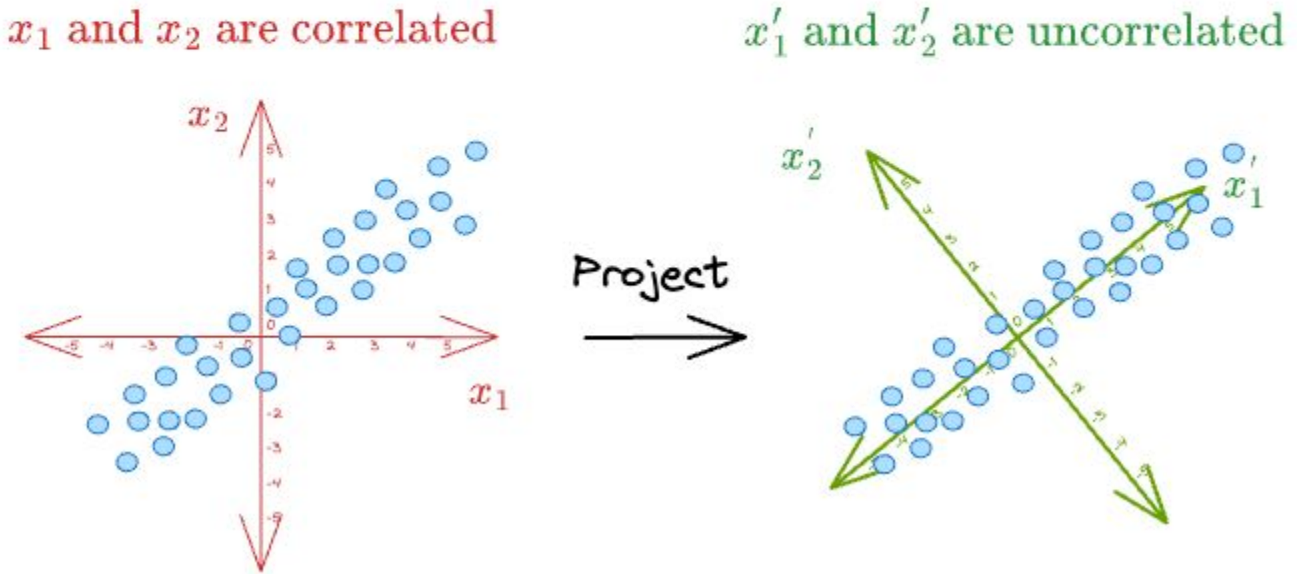


correlated data in native 2D space

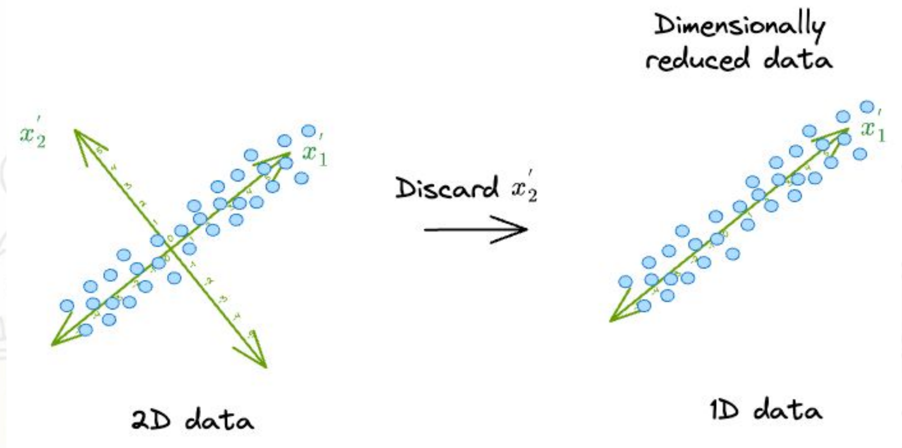


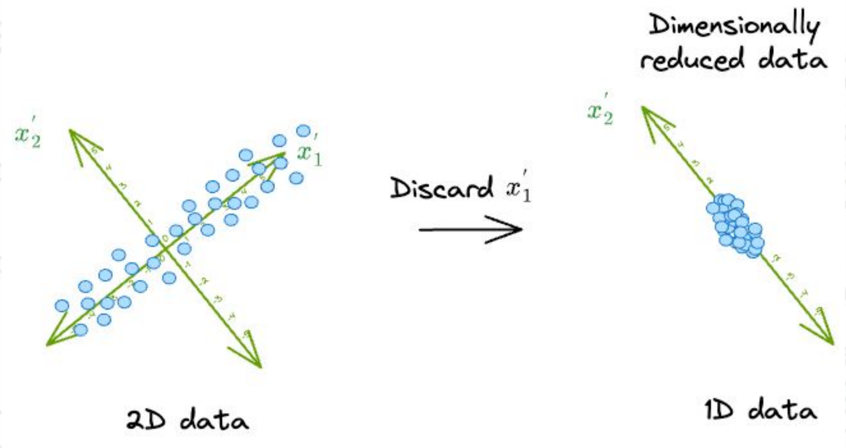
decorrelated data in 2D PC space

(note how there is no statistical information in PC2 - we can ignore it!)



x_1 and x_2 are correlated
x_2
x_1
Project
x'_1 and x'_2 are uncorrelated
x'_2
x'_1


Dimensionally reduced data
x_2'
x_1'
Discard x_2'
2D data
1D data

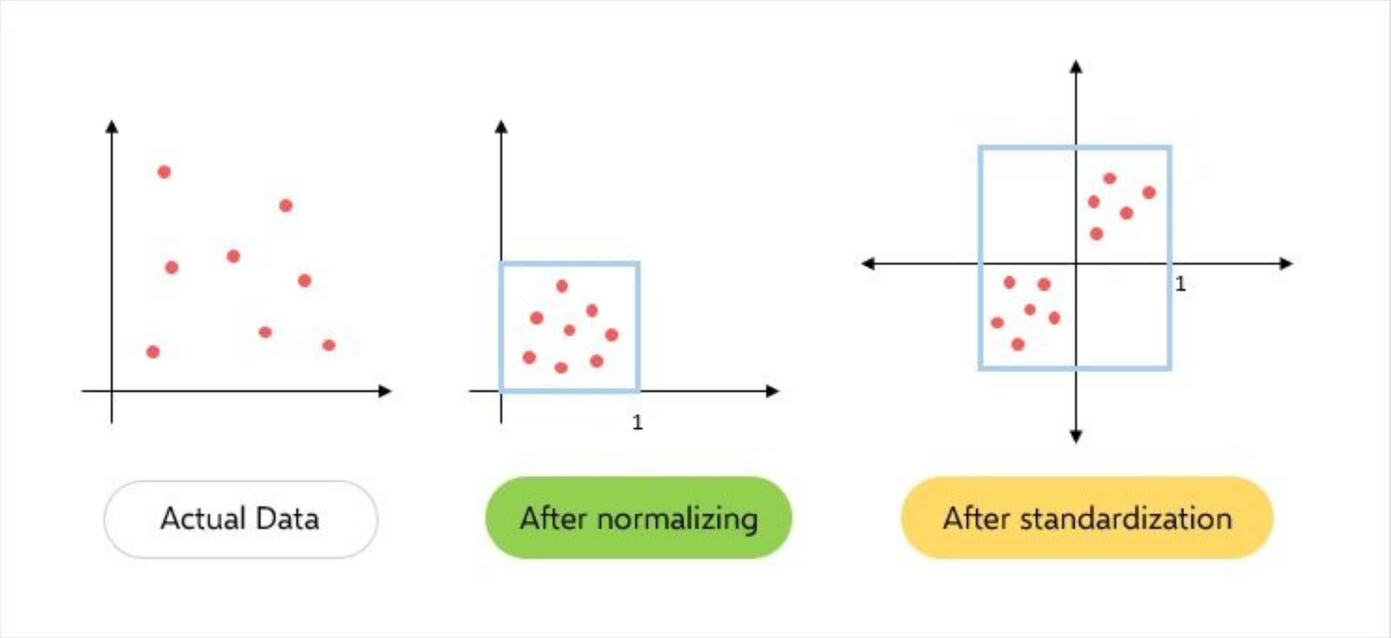

Dimensionally reduced data
x_2'
x_1'
Discard x_1'
2D data
1D data

PCA Transformed Dataset

sepal_len	sepal_wid	petal_len	petal_wid	class
6.7	3.0	5.2	2.3	Iris-virginica
6.3	2.5	5.0	1.9	Iris-virginica
6.5	3.0	5.2	2.0	Iris-virginica
6.2	3.4	5.4	2.3	Iris-virginica
5.9	3.0	5.1	1.8	Iris-virginica

	principal component 1	principal component 2	species
0	-2.264542	-0.505704	Iris-setosa
1	-2.086426	0.655405	Iris-setosa
2	-2.367950	0.318477	Iris-setosa
3	-2.304197	0.575368	Iris-setosa
4	-2.388777	-0.674767	Iris-setosa

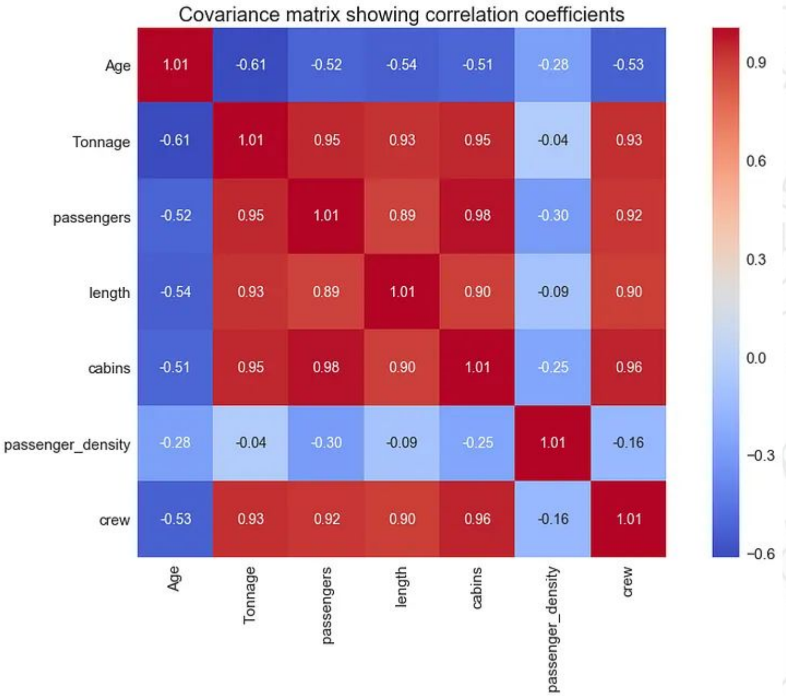
Step 1: Standardardization



Step 2: Calculate Covariance Matrix

$$\begin{bmatrix} Var_1 & Cov_{1,2} & Cov_{1,3} \\ Cov_{1,2} & Var_2 & Cov_{2,3} \\ Cov_{1,3} & Cov_{2,3} & Var_3 \end{bmatrix}$$

$$Cov(x,y) = \frac{\Sigma(x_i-\bar{x})(y_i-\bar{y})}{N}$$



Step 3: Eigen Decomposition

$$A = \mathrm{Cov}(X) = \begin{bmatrix} \mathrm{cov}(X_1, X_1) & \mathrm{cov}(X_1, X_2) \\ \mathrm{cov}(X_2, X_1) & \mathrm{cov}(X_2, X_2) \end{bmatrix} = \begin{bmatrix} \mathrm{cov}(\mathrm{pclass}, \mathrm{pclass}) & \mathrm{cov}(\mathrm{pclass}, \mathrm{fare}) \\ \mathrm{cov}(\mathrm{fare}, \mathrm{pclass}) & \mathrm{cov}(\mathrm{fare}, \mathrm{fare}) \end{bmatrix} = \begin{bmatrix} 2 & 15 \\ 15 & 60 \end{bmatrix}$$

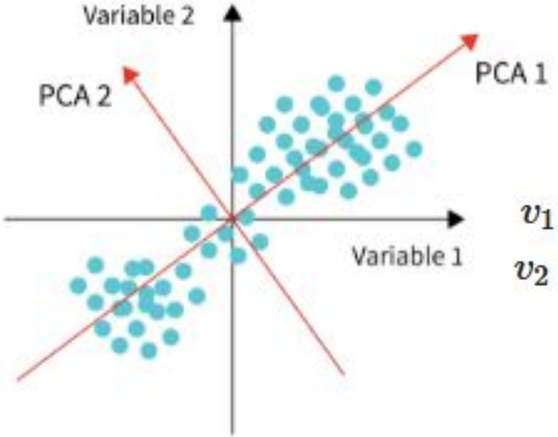
$$A\mathbf{v} = \lambda \mathbf{v}$$

$$\begin{bmatrix} 2 & 15 \\ 15 & 60 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \lambda \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$$

$$A \rightarrow \text{covariance matrix}$$

$$\mathbf{v} \rightarrow \text{eigenvector}$$

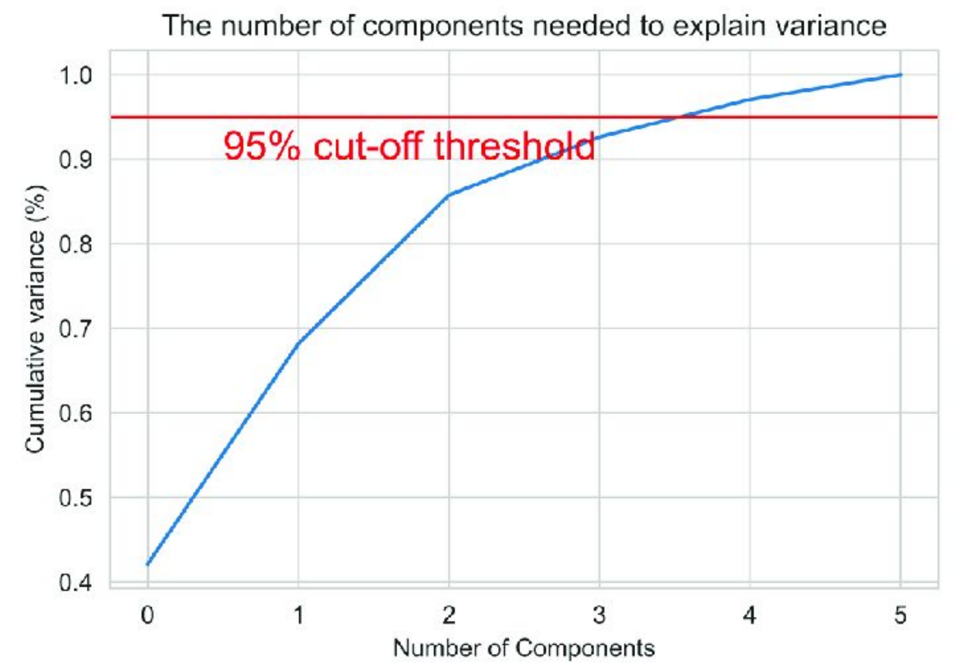
$$\lambda \rightarrow \text{eigenvalue}$$



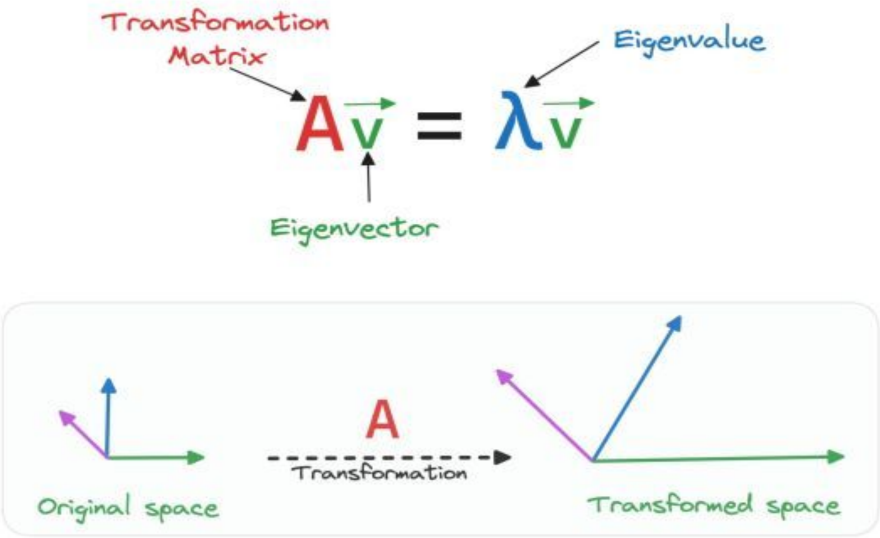
$$v_1 = pc_1 \rightarrow \text{1st principal component}$$

$$v_2 = pc_2 \rightarrow \text{2nd principal component}$$

Step 4: Decide number of components to keep

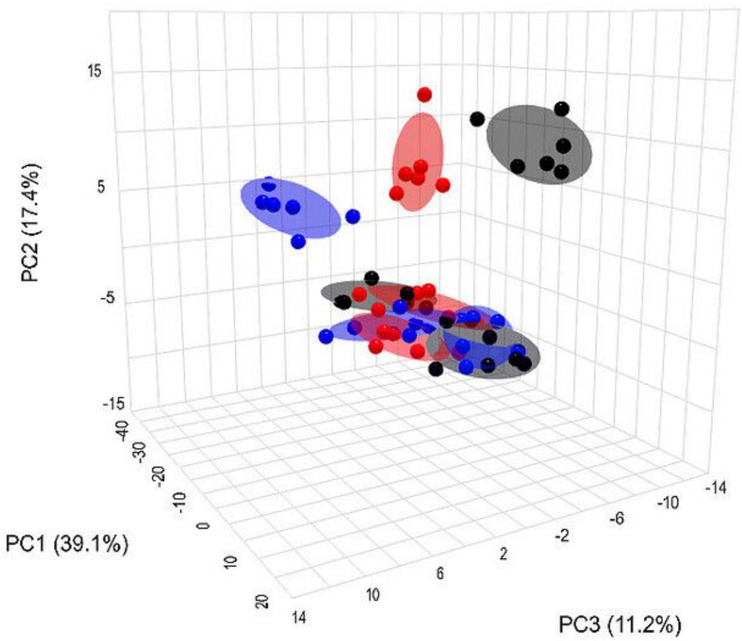


Eigenvector and Eigenvalue



The pink and green vectors are eigenvectors.

When transformed by A, they only scale (by λ) but the direction remains same.



15
5
-5
-15
PC2 (17.4%)
PC1 (39.1%)
PC3 (11.2%)
-40
-30
-20
-10
0
10
20
14
10
6
2
-2
-6
-10
-14

Thank you!